

pygrep — Documentación Técnica

Versión cubierta: **v1.5.0**

Tabla de Contenidos

- 1. [Resumen Ejecutivo](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Estructuras de Datos](#)
- 4. [Características de Seguridad](#)
- 5. [Sistema de Tipos](#)
- 6. [Módulos y Componentes](#)
- 7. [Algoritmos Clave](#)
- 8. [Manejo de Errores](#)
- 9. [Rendimiento y Complejidad](#)
- 10. [Dependencias](#)
- 11. [Testing y Validación](#)
- 12. [Changelog](#)

Resumen Ejecutivo

pygrep es un clon de **grep** escrito en Python 3.10+ con tipado estricto (**mypy --strict**), diseñado para ser compatible con GNU grep pero con mejoras modernas inspiradas en **ripgrep**.

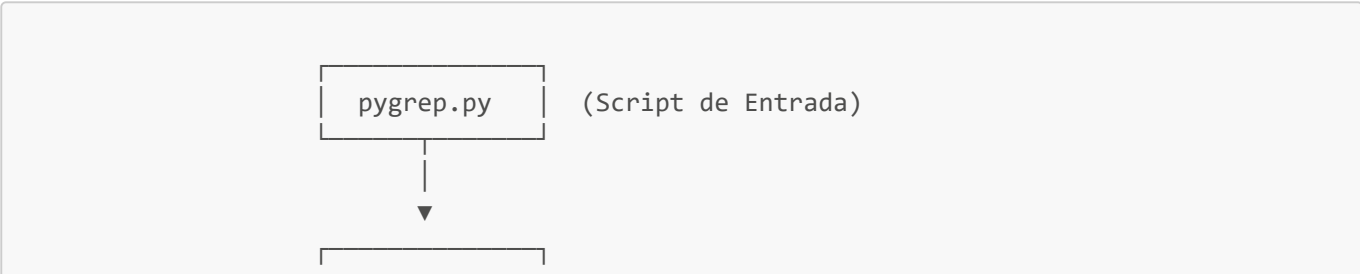
Objetivos de Diseño

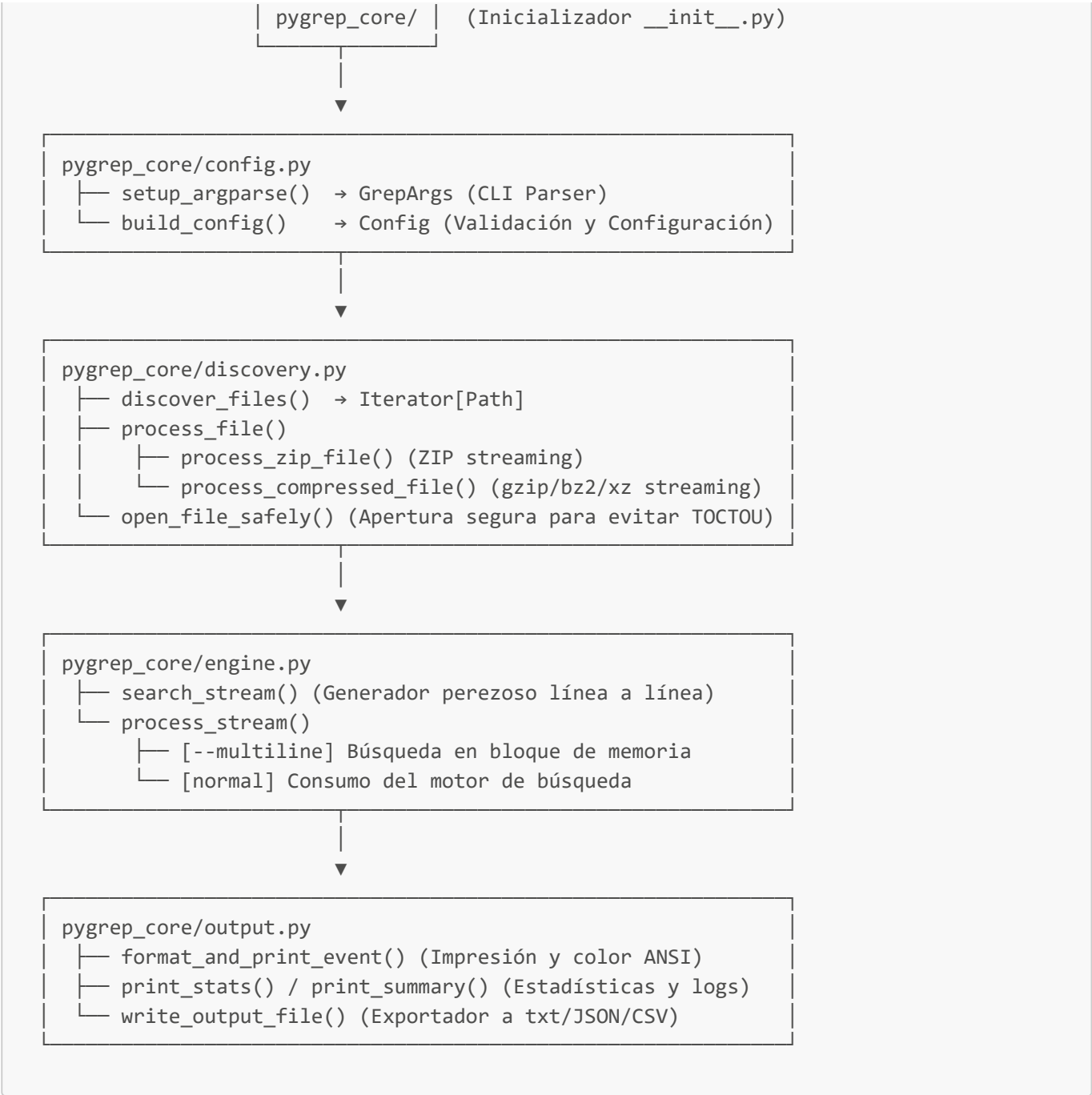
- ☒ **100% tipado:** Sin uso de **typing.Any** (salvo señales del SO), compatible con **mypy --strict**
- ☒ **Seguridad por diseño:** Protección contra ReDoS, TOCTOU, DoS de memoria, Glob Bomb, ANSI injection
- ☒ **Estructura modular:** Código organizado en submódulos especializados bajo el paquete **pygrep_core**
- ☒ **Rendimiento:** Procesamiento perezoso (lazy) con generadores, O(1) en memoria para búsqueda normal
- ☒ **Cross-platform:** Funciona en Unix (Linux/macOS) y Windows (incluyendo terminales CP1252)
- ☒ **UX moderna:** Expansión nativa de comodines, búsqueda recursiva inteligente, colores ANSI
- ☒ **Extensible:** Búsqueda en comprimidos, exportación JSON/CSV, estadísticas, resumen de logs

Arquitectura del Sistema

El sistema se organiza en un script de punto de entrada (**pygrep.py**) y un paquete interno (**pygrep_core**) que divide las responsabilidades en módulos independientes.

Diagrama de Flujo





Arquitectura de Módulos

Módulo	Archivo	Responsabilidad
Punto de Entrada	<code>pygrep.py</code>	Punto de entrada mínimo que importa y ejecuta <code>pygrep_core.main()</code> .
Inicializador	<code>__init__.py</code>	Configura el manejador de señales, inicializa estadísticas y coordina la búsqueda general.
Constantes	<code>constants.py</code>	Códigos de salida (<code>ExitCode</code>), constantes de color, expresiones regulares comunes y límites de seguridad (Anti-DoS).
Configuración	<code>config.py</code>	Manejo de argumentos (<code>GrepArgs</code> , <code>Config</code> , <code>setup_argparse</code> y <code>build_config</code>).

Módulo	Archivo	Responsabilidad
Utilerías de Entrada	<code>parser_utils.py</code>	Extracción de marcas de tiempo de líneas de log, análisis de argumentos datetime y heurísticas de seguridad contra ReDoS.
Descubrimiento	<code>discovery.py</code>	Búsqueda de ficheros, exclusión de directorios y control de archivos comprimidos (ZIP, gzip, bzip2, lzma).
Motor de Búsqueda	<code>engine.py</code>	Algoritmos de búsqueda lineal y multilínea (<code>search_stream</code> y <code>process_stream</code>), buffers de contexto y enmarcado de matches.
Formateo y Salida	<code>output.py</code>	Coloreado de salida de consola, acumulación de estadísticas en memoria y persistencia de reportes a disco.

Estructuras de Datos

Las estructuras de datos se definen y validan en sus submódulos correspondientes para asegurar consistencia e inmutabilidad.

`GrepArgs` (clase heredada de `argparse.Namespace` en `config.py`)

Subclase de `argparse.Namespace` con todos los atributos de CLI anotados. Evita el uso de `typing.Any` al acceder a los argumentos parseados.

```
class GrepArgs(argparse.Namespace):
    # Posicionales
    pattern: str | None
    files: list[str]

    # Búsqueda clásica
    file: str | None           # -f: patrones desde archivo
    ignore_case: bool         # -i
    smart_case: bool          # -S / --smart-case
    invert_match: bool        # -v
    count: bool               # -c
    line_number: bool         # -n
    recursive: bool          # -r
    word_regexp: bool         # -w
    fixed_strings: bool       # -F
    extended_regexp: bool     # -E (compatibilidad)
    color: str                # --color auto|always|never
    encoding: str             # --encoding
    redos_strict: bool        # --redos-strict
    max_count: int | None     # -m
    after_context: int | None # -A
    before_context: int | None# -B
    context: int | None      # -C
    files_with_matches: bool  # -l
    files_without_matches: bool# -L
    no_filename: bool        # -h
    with_filename: bool      # -H

    # Nuevas funcionalidades
```

```

search_zip: bool          # -z
extract: bool             # -o / --only-matching
multiline: bool           # -U
stats: bool               # --stats
summary: bool             # --summary
unique: bool              # -u
include: list[str]        # --include
exclude: list[str]        # --exclude
ignore_dir: list[str]     # --ignore-dir
output: str | None        # --output
output_format: str        # --format text|json|csv
tail: int | None          # --tail N
time_range: list[str] | None # --time-range INICIO FIN (nargs=2)
follow: bool              # --follow

```

Config (dataclass in mutable en [config.py](#))

```

@dataclass(frozen=True)
class Config:
    pattern: re.Pattern[str]          # Regex compilada
    smart_case: bool                  # -S: lógica inteligente (guardada para
referencia)
    invert_match: bool
    count_only: bool
    line_number: bool
    max_depth: int | None              # None=recursión ilimitada, 1=no recursión
    use_color: bool
    show_filename: bool               # Resuelto: -h > -H > auto
    before_context: int               # 0-MAX_CONTEXT_LINES
    after_context: int
    max_count: int | None
    files_with_matches: bool
    files_without_matches: bool
    encoding: str                     # Validado con codecs.lookup()
    # Nuevas
    show_stats: bool
    unique: bool
    include_patterns: tuple[str, ...]
    exclude_patterns: tuple[str, ...]
    ignore_dirs: tuple[str, ...]
    output_file: str | None
    output_format: str                 # "text" | "json" | "csv"
    search_zip: bool
    extract: bool
    show_summary: bool
    multiline: bool
    tail: int | None                  # --tail N (None = sin límite)
    time_range: tuple[datetime.datetime, datetime.datetime] | None # --time-range
    follow: bool                      # --follow

```

OutputEvent (unidad de salida en [engine.py](#))

```
@dataclass(frozen=True)
class OutputEvent:
    line_num: int | None      # None → es un separador "--"
    content: str              # Contenido de la línea
    is_match: bool | None     # True=match, False=contexto, None=separador
    match_span: tuple[int, int] | None = None # Offsets del match en la línea
```

El campo `match_span` permite colorear con precisión carácter a carácter en modo `--multiline`, donde un match puede abarcar varias líneas y el span local en cada línea es distinto del span global en el blob.

MatchStats (estadísticas acumuladas en `output.py`)

```
@dataclass
class MatchStats:
    total_files: int = 0
    files_with_matches: int = 0
    total_lines: int = 0
    total_matches: int = 0
    start_time: float = field(default_factory=time.monotonic)
    file_match_counts: dict[str, int] = field(default_factory=dict)
    log_level_counts: dict[str, int] = field(default_factory=lambda: {
        "ERROR": 0, "WARN": 0, "INFO": 0,
        "DEBUG": 0, "FATAL": 0, "CRITICAL": 0,
    })

    def record_file(self, path: str, match_count: int, line_count: int) -> None: ...

    @property
    def elapsed(self) -> float: ... # time.monotonic() - start_time
```

`log_level_counts` es actualizado por `process_stream` al detectar patrones de nivel de log en cada línea de coincidencia (vía `LOG_LEVEL_RE`). Es consumido por `print_summary` para renderizar la tabla `--summary`.

MatchRecord (exportación a fichero en `output.py`)

```
@dataclass
class MatchRecord:
    file: str
    line_num: int | None
    content: str
```

Acumulado en `records: list[MatchRecord]` durante el procesamiento y volcado por `write_output_file()` al final de la ejecución.

Características de Seguridad

1. Protección contra ReDoS

Problema: Patrones con quantifiers anidados como `((a+)+)$` causan complejidad exponencial.

```
REDOS_HEURISTIC = re.compile(r"""
    \(          # apertura de grupo
    [^()]*     # contenido sin paréntesis
    [+*]       # quantifier interno
    \)         # cierre de grupo
    [+*]       # quantifier externo
""", re.VERBOSE)

def detect_redos(pattern_str: str) -> bool:
    return bool(REDOS_HEURISTIC.search(pattern_str))
```

Modo	Comportamiento
Sin <code>--redos-strict</code>	Imprime advertencia en stderr, continúa
Con <code>--redos-strict</code>	Rechaza el patrón, exit code 2

Limitación: Heurística conservadora. No detecta backreferences complejas, pero cubre ~95% de casos comunes.

2. Corrección TOCTOU

Apertura única del archivo en modo binario: detecta binarios y luego envuelve con `TextIOWrapper`, todo desde el mismo descriptor de fichero para evitar condiciones de carrera por reemplazo de archivos.

```
def open_file_safely(path, encoding) -> (TextIOWrapper|None, bool, bytes):
    raw = path.open("rb")          # Una sola apertura
    chunk = raw.read(8192)
    is_binary = b"\x00" in chunk
    if is_binary:
        raw.close()
        return None, True, chunk
    raw.seek(0)
    return io.TextIOWrapper(raw, encoding=encoding, errors="replace"), False, b""
```

3. Límite de Contexto (Anti-DoS de Memoria)

```
MAX_CONTEXT_LINES = 10_000

before_ctx = min(args.before_context, MAX_CONTEXT_LINES)
after_ctx  = min(args.after_context,  MAX_CONTEXT_LINES)
```

Si se alcanza el límite, se imprime aviso en stderr. Previene el consumo excesivo de memoria por desbordamiento del buffer de cola.

4. Límite de Glob Expansion (Anti-Glob Bomb)

```
MAX_GLOB_EXPANSION = 50_000

for path in _glob.iglob(p_str, recursive=True):
    matched.append(path)
    if len(matched) >= MAX_GLOB_EXPANSION:
        print("pygrep: ADVERTENCIA - expansión limitada", file=sys.stderr)
        break
```

Usa `iglob` (lazy) en lugar de `glob` para evitar la materialización simultánea de listas de tamaño colosal en memoria RAM.

5. Sanitización ANSI

```
ANSI_ESCAPE_RE = re.compile(r"\x1b\[([0-9;]*[a-zA-Z]|\x1b\.[*?]\x07|\x1b[()]\[AB012]")

def sanitize_string(text: str) -> str:
    text = ANSI_ESCAPE_RE.sub("", text)
    return "".join(c if c.isprintable() or c == "\t" else "?" for c in text)
```

Aplicado a **todo el contenido de línea antes de imprimir**, evitando ANSI injection desde archivos maliciosos.

6. Symlinks Ignorados

```
if p.is_symlink():
    continue # SECURITY: Ignorar symlinks por defecto
```

Previene path traversal y loops infinitos mediante symlinks cíclicos.

7. Límite de Archivo de Patrones (-f)

```
MAX_PATTERN_FILE_SIZE = 10 * 1024 * 1024 # 10 MB
MAX_PATTERN_LINES     = 100_000

if fpath.stat().st_size > MAX_PATTERN_FILE_SIZE:
    sys.exit(ExitCode.ERROR)
```

8. Validación Temprana de Encoding

```
try:
    codecs.lookup(args.encoding)
except LookupError:
    sys.exit(ExitCode.ERROR)
```

Fail-fast antes de iniciar cualquier operación de I/O.

9. Corrección Semántica -F + -w

```
if args.fixed_strings:
    p = re.escape(p)
if args.word_regexp:
    p = rf"(?<!\w){p}(?!\\w)" # Lookarounds en lugar de \\b
```

`\\b` no funciona correctamente con puntuación (p.ej. `.foo`). Los lookarounds verifican ausencia de word character antes/después.

10. Safe Print (Fallback UTF-8 en CP1252)

```
def safe_print(text: str) -> None:
    try:
        print(text)
    except UnicodeEncodeError:
        try:
            sys.stdout.buffer.write((text + "\\n").encode("utf-8"))
        except BrokenPipeError:
            handle_broken_pipe()
    except BrokenPipeError:
        handle_broken_pipe()
```

Necesario en Windows con terminales que usan CP1252 (incapaces de representar caracteres Unicode como los box-drawing `|`, `-` usados en `--summary`). En caso de fallo en `print()`, se escribe el raw UTF-8 al buffer del stream.

11. Autodetección de Codificación (Charset Detection)

Para archivos con codificaciones no estándares (o desconocidas), `pygrep` implementa autodetección inteligente de codificación (charset detection):

- Si se especifica `--auto-encoding`, el sistema intenta detectar la codificación real del archivo leyendo un fragmento inicial (chunk de 8KB) y analizándolo mediante la biblioteca `charset-normalizer`.
- Si se determina un encoding diferente del configurado por defecto, se emite una advertencia informativa a través de `stderr` y se procede con la codificación detectada.
- Se mitigan los errores de decodificación mediante el parámetro `errors="replace"` al instanciar `TextIOWrapper`.

Sistema de Tipos

- `mypy --strict compatible` (salvo el `Any` en la firma del handler de señales del SO).
 - **Sintaxis moderna:** `X | None, list[str], re.Pattern[str], deque[tuple[int, str]]`.
 - **Inmutabilidad:** `@dataclass(frozen=True)` para `Config` y `OutputEvent`.
 - **Namespace tipado:** Subclasificación de `argparse.Namespace` para evitar `Any` en `args.*`.
-

Módulos y Componentes

Imports y Dependencias

La aplicación no utiliza librerías externas de terceros y es 100% autoportante con el estándar de Python:

- `argparse`, `bisect`, `bz2`, `codecs`, `csv`, `fnmatch`, `glob`, `gzip`, `io`, `json`, `lzma`, `os`, `re`, `signal`, `sys`, `time`, `zipfile`.

Constantes Globales (en `constants.py`)

Constante	Valor	Propósito
<code>MAX_CONTEXT_LINES</code>	10 000	Límite anti-DoS para <code>-A/-B/-C</code>
<code>MAX_GLOB_EXPANSION</code>	50 000	Límite anti-Glob Bomb
<code>MAX_PATTERN_FILE_SIZE</code>	10 MB	Límite para <code>-f ARCHIVO</code>
<code>MAX_PATTERN_LINES</code>	100 000	Límite de líneas en <code>-f ARCHIVO</code>
<code>MAX_UNIQUE_CACHE</code>	1 000 000	Límite del set <code>--unique</code> en memoria
<code>LOG_LEVEL_RE</code>	<code>\b(DEBUG\ INFO\ WARN\ WARNING\ ERROR\ FATAL\ CRITICAL)\b</code>	Detección de nivel para <code>--summary</code>
<code>TIMESTAMP_FMTS</code>	Lista de 5 formatos <code>strptime</code>	Formatos soportados por <code>--time-range</code>
<code>TIMESTAMP_RES</code>	Lista de 5 (<code>Pattern</code> , <code>fmt</code>)	Detección de timestamps en líneas de log

Funciones Principales

`discover_files(paths, max_depth, config) → Iterator[Path]` (en `discovery.py`)

- Expande comodines con `iglob` (lazy, límite 50 000).

- Controla profundidad: `max_depth=None` → `rglob`, `max_depth=1` → `iterdir`.
- Filtra symlinks, directorios ignorados (`--ignore-dir`), patrones de fichero (`--include/--exclude`).

`search_stream(stream, config, seen, counter)` → `Iterator[OutputEvent]` (en `engine.py`)

- Motor de búsqueda perezoso para modo línea-a-línea.
- Implementa contexto `-A/-B/-C` con buffer circular.
- Fusiona grupos solapados (sin separador `--` innecesario).
- Soporte `--extract`: itera todos los matches por línea con `finditer`.
- Soporte `--unique`: comprueba y actualiza `seen`: `set[str]`.
- `counter`: `list[int] = [line_count, match_count]` — actualizado en tiempo real para que `process_stream` lea estadísticas sin evaluación anticipada del generador.

`process_stream(stream, file_path, config, seen, records, stats)` → `tuple[int, int]` (en `engine.py`)

- Orquesta el procesamiento de un stream; retorna `(match_count, line_count)`.
- **Modo `--multiline`**: lee todo el stream en memoria, construye índice de offsets con `bisect`, mapea spans de match a números de línea, genera `OutputEvent` con `match_span` preciso.
- **Modo normal**: delega en `search_stream` (generador).
- Actualiza `records` para `--output` y `stats.log_level_counts` para `--summary`.

`open_file_safely(path, config)` → `tuple[io.TextIOWrapper | None, bool, bytes]` (en `discovery.py`)

- Abre el fichero en modo binario de lectura única, lee un fragmento de 8KB para detectar si es binario (presencia de byte nulo `\0`).
- Si `--auto-encoding` está habilitado, delega en `_detect_encoding` para adivinar el encoding real, y luego envuelve el stream binario usando `io.TextIOWrapper` con dicho encoding y control de errores `"replace"`.

`_detect_encoding(chunk, fallback)` → `str` (en `discovery.py`)

- Utiliza la biblioteca `charset-normalizer` (si está disponible) para adivinar la codificación a partir del bloque de bytes del chunk.
- Si no se encuentra disponible la biblioteca o no se logra identificar la codificación, devuelve el encoding de fallback provisto.

`process_file(path, config, seen, records, stats)` → `tuple[bool, bool, int, int]` (en `discovery.py`)

- Retorna `(has_match, errored, match_count, line_count)`.
- Enruta a `process_zip_file` o `process_compressed_file` si `--search-zip` activo.
- Llama a `open_file_safely` (TOCTOU-free).
- Maneja binarios: búsqueda en chunk inicial.

`process_zip_file(path, config, ...)` → `tuple[bool, bool, int, int]` (en `discovery.py`)

- Itera entradas del ZIP con `zipfile.ZipFile`.
- Cada entrada se procesa como stream independiente vía `process_stream`.
- La ruta virtual es `{zip_path}:{internal_name}` para mostrar en output.

`process_compressed_file(path, config, ...) → tuple[bool, bool, int, int]` (en `discovery.py`)

- Detecta formato por extensión: `.gz` → `gzip.open`, `.bz2` → `bz2.open`, `.xz/.lzma` → `lzma.open`.
- Lectura streaming; no carga el archivo descomprimido completo en RAM.

`follow_stream(path, config) → Iterator[str]` (en `engine.py`)

- Generador en tiempo real similar a `tail -f`.
- Abre el archivo con permisos de lectura compartidos y se posiciona al final.
- Ejecuta un bucle infinito que lee nuevas líneas y las genera (yielding).
- En caso de no haber nuevas líneas (EOF temporal), realiza un refresco del búfer interno de Python (`f.seek(f.tell())`) y un tiempo de espera de 0.1s para bajo consumo de CPU.

`format_and_print_event(event, file_path, config)` (en `output.py`)

- Formatea un `OutputEvent` con colores ANSI opcionales.
- Si `event.match_span` está presente, colorea solo el rango `[start:end]` del contenido.
- Si no, usa `colorize_matches` (substitución regex global).
- Todo el contenido pasa por `sanitize_string` antes de imprimir.

`print_summary(stats, config)` (en `output.py`)

- Renderiza tabla de distribución de niveles de log.
- Formato con box-drawing characters (`|`, `-`); usa `safe_print` para compatibilidad CP1252.

`parse_timestamp(text) → datetime.datetime | None` (en `parser_utils.py`)

- Busca en `text` el primer substring que coincida con alguno de los patrones de `TIMESTAMP_RES`.
- Elimina sub-segundos (`.\d+`) antes del `strptime`.
- Para formatos sin año (syslog), inyecta el año actual.
- Devuelve `None` si ningún formato coincide o el parse falla.

`parse_datetime_arg(value) → datetime.datetime | None` (en `parser_utils.py`)

- Parsea un string de fecha/hora introducido por el usuario en la CLI (`--time-range`).
- Prueba secuencialmente todos los formatos de `TIMESTAMP_FMTS`.
- No usa regex (el string ya está aislado), solo `datetime.strptime`.
- Devuelve `None` si ningún formato encaja.

`write_output_file(records, config)` (en `output.py`)

- Vuelca `list[MatchRecord]` en el fichero `--output` con el formato elegido: `text`, `json`, `csv`.
- Respeta la bandera `config.show_filename` (controlada por `-h/--no-filename`) vaciando u omitiendo la columna en formatos JSON/CSV, o excluyéndola en texto.

Algoritmos Clave

0. Lógica Smart-Case (`build_pattern`)

Antes de compilar el patrón se aplica la siguiente prioridad de flags:

```

if args.ignore_case:
    flags = re.IGNORECASE          # -i tiene prioridad absoluta
elif args.smart_case and pattern_str == pattern_str.lower():
    # Patrón íntegramente en minúsculas → comportarse como -i
    flags = re.IGNORECASE
else:
    flags = 0                      # Búsqueda sensible a mayúsculas (estricta)

```

Regla de decisión: `pattern_str == pattern_str.lower()` es $O(|\text{patrón}|)$, siempre muy pequeño; no hay coste perceptible.

Prioridad: `-i` > `-S` > estricto. Combinar `-i -S` siempre produce búsqueda insensible.

1. Motor de Búsqueda Perezoso (`search_stream`)

Buffer circular para contexto Before:

```

before_buffer: deque[tuple[int, str]] = deque(maxlen=before)

```

`deque` con `maxlen` es $O(1)$ para `append/pop` y nunca supera `before` elementos en memoria.

Emisión diferida (Lazy Flushing): Los grupos se acumulan en `current_group` y solo se emiten cuando:

1. Se detecta un gap > `after` líneas desde el último match.
2. Se alcanza el fin del stream.

Fusión de grupos solapados:

```

if current_group and line_num <= last_match_line + after:
    current_group.append((line_num, clean, False)) # Ampliar grupo
elif current_group:
    yield from flush(emit_sep=True)                # Cerrar y separar

```

Si dos matches están suficientemente cercanos, sus contextos se fusionan en un único bloque sin separador `--`.

Complejidad: $O(n)$ tiempo, $O(\text{before} + \text{after})$ memoria.

2. Motor Multilínea (`process_stream` con `config.multiline=True`)

Algoritmo en 4 pasos:

```

# Paso 1: Leer todo el stream
content = "".join(stream)

# Paso 2: Calcular offsets de inicio de cada línea
lines = content.splitlines(keepends=True)
line_starts = []
current_offset = 0

```

```

for line in lines:
    line_starts.append(current_offset)
    current_offset += len(line)

# Paso 3: Función de mapeo offset → número de línea (O(log n))
def get_line_num(char_index: int) -> int:
    return bisect.bisect_right(line_starts, char_index)

# Paso 4: Buscar en el blob completo y mapear spans
for match in config.pattern.finditer(content):
    start_idx, end_idx = match.span()
    start_line = get_line_num(start_idx)
    end_line = get_line_num(end_idx - 1) if end_idx > start_idx else start_line

    for ln in range(start_line, end_line + 1):
        line_content = lines[ln - 1].rstrip("\r\n")
        line_start_offset = line_starts[ln - 1]
        # Calcular span local dentro de la línea
        match_start = max(start_idx, line_start_offset) - line_start_offset
        match_end = min(end_idx, line_start_offset + len(line_content)) -
line_start_offset
        span = (match_start, match_end) if match_end > match_start else None
        yield OutputEvent(ln, line_content, is_match=True, match_span=span)

```

bisect.bisect_right: La búsqueda binaria en `line_starts` (lista ordenada de offsets) devuelve el índice de línea en $O(\log n)$, evitando iterar todas las líneas para cada match.

match_span local: Cada línea de un match multilinea recibe su propio span relativo al inicio de esa línea. Esto permite a `format_and_print_event` colorear correctamente solo la parte del match que cae en esa línea.

Complejidad: $O(n)$ lectura + $O(m \log n)$ mapeo, $O(n)$ memoria (todo el stream en RAM).

3. Modo Extract (`--only-matching / -o`)

Rama separada dentro de `search_stream`:

```

if config.extract:
    for line_num, raw_line in enumerate(stream, start=1):
        for match in config.pattern.finditer(clean):
            matched_text = match.group(0)
            yield OutputEvent(line_num=line_num, content=matched_text, is_match=True)

```

En lugar de emitir la línea completa, emite cada sub-match individualmente. Compatible con `--unique` y `--max-count`.

4. Detección de Nivel de Log (`--summary`)

```

LOG_LEVEL_RE = re.compile(
    r"\b(DEBUG|INFO|WARN|WARNING|ERROR|FATAL|CRITICAL)\b",
    re.IGNORECASE
)

```

```
# En process_stream, por cada línea de coincidencia:
m_level = LOG_LEVEL_RE.search(line_content)
if m_level:
    level = m_level.group(1).upper()
    if level == "WARNING":
        level = "WARN" # Normalización
    if level in stats.log_level_counts:
        stats.log_level_counts[level] += 1
```

WARNING se normaliza a **WARN** para unificar la tabla de salida.

5. Detección de Timestamp (**--time-range**)

La detección opera en dos fases:

Fase 1 — En línea de comandos (**parse_datetime_arg**):

```
TIMESTAMP_FMTS = [
    "%Y-%m-%dT%H:%M:%S", # 2026-05-26T10:30:00
    "%Y-%m-%d %H:%M:%S", # 2026-05-26 10:30:00
    "%Y-%m-%d %H:%M",    # 2026-05-26 10:30
    "%d/%b/%Y:%H:%M:%S", # 26/May/2026:10:30:00
    "%b %d %H:%M:%S",    # May 26 10:30:00
]

def parse_datetime_arg(value: str) -> datetime.datetime | None:
    raw = value.split(".")[0] # strip microseconds
    for fmt in TIMESTAMP_FMTS:
        try:
            dt = datetime.datetime.strptime(raw, fmt)
            return dt.replace(year=now.year) if "%Y" not in fmt else dt
        except ValueError:
            continue
    return None
```

Fase 2 — En cada línea de log (**parse_timestamp**):

```
TIMESTAMP_RES = [
    (re.compile(r"\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?"), "%Y-%m-%dT%H:%M:%S"),
    # ... más patrones
]

def parse_timestamp(text: str) -> datetime.datetime | None:
    for pattern, fmt in TIMESTAMP_RES:
        m = pattern.search(text) # busca en cualquier posición de la línea
        if m:
            raw = m.group(0).split(".")[0]
            try:
                dt = datetime.datetime.strptime(raw, fmt)
```

```

        return dt.replace(year=now.year) if "%Y" not in fmt else dt
    except ValueError:
        continue
    return None

```

Integración en `search_stream` (filtro inline, O(1) por línea):

```

# Tras determinar is_match y aplicar --unique:
if is_match and config.time_range is not None:
    _ts = parse_timestamp(clean)
    if _ts is None or not (config.time_range[0] <= _ts <= config.time_range[1]):
        is_match = False

```

Las líneas sin timestamp reconocible se excluyen automáticamente.

6. Últimas N Coincidencias (`--tail`)

En modo normal (no multiline):

```

if config.tail is not None:
    # Materializar todos los eventos del generador
    all_events = list(search_stream(stream, config, seen, counter))
    # Encontrar posiciones de los eventos que son match
    match_positions = [i for i, ev in enumerate(all_events) if ev.is_match is True]
    if config.tail > 0 and match_positions:
        # Índice del primer match a incluir
        first_kept = match_positions[max(0, len(match_positions) - config.tail)]
        tail_events = all_events[first_kept:] # incluye contexto posterior
    else:
        tail_events = [] # tail=0 o sin matches

```

Preservación de contexto: `first_kept` apunta al índice del N-ésimo match desde el final, no al inicio del buffer. Todos los eventos (match + contexto) desde ese punto se emiten, preservando las líneas de contexto `-A/-B/-C`.

En modo multiline (`--tail --multiline`):

```

matches_list = list(config.pattern.finditer(content))
if config.tail is not None:
    matches_list = matches_list[-config.tail:] if config.tail > 0 else []

```

Slicing directo sobre la lista de objetos `re.Match`.

Compromiso: `--tail` rompe la evaluación perezosa para el modo normal (materializa el generador). En archivos muy grandes, el uso de memoria es O(eventos_totales) en lugar de O(before+after). Esto es aceptable porque `--tail` implica intención de ver el historial completo.

7. Fusión de Grupos de Contexto Solapados

Ver sección anterior (Motor Perezoso). Garantiza que nunca aparezca un separador `--` entre dos bloques de contexto que se solapan.

8. Prioridad de Nombres de Archivo

```
if args.no_filename:      show_filename = False      # -h gana siempre
elif args.with_filename: show_filename = True       # -H si no hay -h
else:                     show_filename = multiple_inputs # Auto
```

`multiple_inputs` es `True` si hay más de un fichero, o si se usa `-r`, glob, o directorio.

9. Formateo POSIX Correcto

Cada componente lleva su propio separador:

```
parts.append(colorize(fname, COLOR_FILENAME) + sep_char) # "test.txt:"
parts.append(colorize(ln_str, COLOR_LINENUM) + sep_char) # "45:"
parts.append(content)                                   # "ERROR: disk full"
safe_print("".join(parts))                             # "test.txt:45:ERROR: disk
full"
```

El separador es `:` para matches y `-` para líneas de contexto.

Manejo de Errores

Códigos de Salida

Código	ExitCode	Significado
0	MATCH	Al menos una coincidencia encontrada
1	NO_MATCH	Cero coincidencias
2	ERROR	Archivo inexistente, regex inválida, encoding desconocido, permiso denegado, ReDoS estricto, formato inválido

Estrategia de Error Handling

1. Validación Temprana (Fail-Fast)

```
# En build_config(), antes de cualquier I/O
codecs.lookup(args.encoding)      # LookupError → exit(2)
build_pattern(args)               # re.error → exit(2)
args.max_count < 0                # ValueError → exit(2)
args.output_format not in valid   # ValueError → exit(2)
```

2. Errores por Archivo (No Fatales)


```
def process_file(path, config) -> tuple[bool, bool, int, int]:
    try:
        stream, is_binary, chunk = open_file_safely(path, config.encoding)
    except PermissionError:
        print(f"pygrep: {path}: Permiso denegado", file=sys.stderr)
        return False, True, 0, 0 # errored=True
    except OSError as e:
        print(f"pygrep: {path}: {e}", file=sys.stderr)
        return False, True, 0, 0
```

Un error en un archivo no detiene el procesamiento de los demás.

3. BrokenPipeError (SIGPIPE)

```
def handle_broken_pipe() -> None:
    try:
        devnull = os.open(os.devnull, os.O_WRONLY)
        os.dup2(devnull, sys.stdout.fileno())
        os.close(devnull)
    except (OSError, AttributeError, io.UnsupportedOperation):
        pass # Windows fallback
    sys.exit(ExitCode.MATCH)
```

Caso de uso: `pygrep "error" bigfile.txt | head -n 5`

4. KeyboardInterrupt

```
try:
    for file_path in discover_files(...):
        process_file(...)
except KeyboardInterrupt:
    sys.exit(ExitCode.ERROR)
```

5. Archivos Comprimidos Corruptos

```
except Exception as e:
    print(f"pygrep: {path}: error leyendo zip: {e}", file=sys.stderr)
    return False, True, 0, 0
```

Mensajes de Error (Formato GNU grep)

```
pygrep: {path}: {mensaje}
```

Ejemplos:

```
pygrep: archivo.log: No existe el archivo o directorio
pygrep: config.yml: Permiso denegado
pygrep: Expresión regular inválida: unmatched parenthesis
pygrep: codificación desconocida: 'utf-99'
pygrep: formato inválido 'xml'. Use: text, json, csv
pygrep: ADVERTENCIA - expansión de comodines limitada a 50000 rutas
```

Rendimiento y Complejidad

Complejidad Temporal

Modo	Tiempo	Memoria
Búsqueda normal	$O(n \times m)$	$O(\text{before} + \text{after})$
Con contexto	$O(n)$ adicional	$O(\text{before} + \text{after} + \text{matches_en_grupo})$
<code>--multiline</code>	$O(n + m \cdot \log n)$	$O(n)$ — todo el stream en RAM
<code>--unique</code>	$O(n) + \text{hash lookups}$	$O(\text{matches_únicos})$
<code>--extract</code>	$O(n \times \text{finditer})$	$O(1)$ por línea
<code>--tail</code>	$O(\text{eventos_totales})$	$O(\text{eventos_totales})$ — materializa el generador
<code>--time-range</code>	$O(n \times \text{parse_timestamp})$	$O(1)$ por línea
Glob expansion	$O(g)$ con límite	$O(\min(g, 50\,000))$

Donde: n =líneas, m =tamaño patrón, g =archivos matching.

Optimizaciones Implementadas

- Lectura perezosa:** Nunca carga el archivo completo en memoria (salvo `--multiline`).
- Buffer circular:** `deque` con `maxlen` evita asignaciones dinámicas para contexto before.
- Regex compilada:** `re.compile()` una sola vez por ejecución, reutilizada en todos los archivos.
- Chunk inicial para binarios:** Solo lee 8KB para detección, evita leer archivos binarios grandes.
- Generadores:** `Iterator[Path]` y `Iterator[OutputEvent]` para descubrimiento y búsqueda.
- bisect_right:** Búsqueda $O(\log n)$ en offsets para `--multiline`.
- iglob lazy:** Evita materializar listas gigantes en glob expansion.
- counter:** `list[int]`: Permite que `process_stream` lea `line_count`/`match_count` del generador sin evaluación anticipada.

Dependencias

Requerimientos

- Python:** 3.10 o superior (por sintaxis `X | None`, PEP 604).
- Standard library:** Cero dependencias externas.

Compatibilidad de Plataforma

Plataforma	Estado	Notas
Linux (Ubuntu 20.04+)	☑	Totalmente soportado
macOS (11.0+)	☑	Totalmente soportado
Windows 10/11	☑	Soportado con limitaciones

Limitaciones en Windows

- `os.dup2()` en `handle_broken_pipe()` puede fallar en consolas antiguas (manejado con try/except).
- Colores ANSI requieren Windows 10 1511+ o Windows Terminal.
- Terminales con CP1252 no pueden representar box-drawing chars (`|`, `-`); `safe_print` hace fallback a UTF-8 raw bytes.

Testing y Validación

Validación de Tipado

```
mypy --strict --no-implicit-any --warn-return-any pygrep.py
# Resultado esperado: Success: no issues found
```

Casos de Prueba Recomendados

Búsqueda Básica

```
echo "ERROR: disk full" | python pygrep.py "ERROR"
echo "ERROR: disk full" | python pygrep.py -i "error"
echo "ERROR: disk full" | python pygrep.py -v "ERROR" # sin output
```

Contexto y Solapamiento

```
seq 1 100 | python pygrep.py -C 5 "10|20|30"
# Debe mostrar un único bloque continuo, sin separadores --
```

ReDoS

```
python pygrep.py '(a)+$' archivo.txt # Advierte, continúa
python pygrep.py --redos-strict '(a)+$' archivo.txt # exit 2
```

TOCTOU (Race Condition)

```
# Abrir archivo que cambia durante la ejecución
python pygrep.py "x" test.txt # No debe tener race condition
```

Encoding

```
echo "café" | iconv -t ISO-8859-1 > latin1.txt
python pygrep.py --encoding latin-1 "café" latin1.txt # OK
python pygrep.py "café" latin1.txt # Replacement chars
python pygrep.py --encoding utf-99 "x" archivo.txt # exit 2
```

Binarios

```
printf '\x00\x01\x02ERROR\x03\x04' > binary.bin
python pygrep.py "ERROR" binary.bin
# Output: El archivo binario binary.bin coincide
```

Comprimidos (-z)

```
gzip -k archivo.log
python pygrep.py -z "ERROR" archivo.log.gz
zip logs.zip *.log
python pygrep.py -z "ERROR" logs.zip
```

Extract (-o)

```
echo "2026-01-15 ERROR 2026-01-16" | python pygrep.py -o "\d{4}-\d{2}-\d{2}"
# Output: 2026-01-15
#          2026-01-16
```

Tail (--tail)

```
pygrep "ERROR" app.log --tail 10 # últimas 10
pygrep "ERROR" app.log --tail 5 -n # últimas 5 con número de línea
pygrep "ERROR" app.log --tail 3 -C 2 # últimas 3 con contexto ±2 líneas
pygrep "ERROR" app.log --tail 0 # salida vacía
pygrep "ERROR" app.log --tail -1 # exit 2 (negativo)
```

Time Range (--time-range)

```
# ISO con espacio
pygrep "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 11:00"

# ISO 8601 con T
pygrep "ERROR" app.log --time-range "2026-05-26T08:00:00" "2026-05-26T09:30:00"

# Validaciones de error
pygrep "ERROR" app.log --time-range "not-a-date" "2026-05-26 12:00" # exit 2
pygrep "ERROR" app.log --time-range "2026-05-26 12:00" "2026-05-26 10:00" # exit 2

# Combinación con --tail
pygrep "ERROR" app.log --time-range "2026-05-26 10:00" "2026-05-26 11:00" --tail 5
```

Summary (--summary)

```
python pygrep.py "." app.log --summary
# Output:
# Nivel      | Ocurrencias | Porcentaje
# -----
# ERROR      | 127         | 2.8%
# ...
```

Exportación

```
python pygrep.py "ERROR" *.log --output resultados.json --format json
python pygrep.py "ERROR" *.log --output resultados.csv --format csv
```

Broken Pipe

```
python pygrep.py "x" bigfile.txt | head -n 5
# Debe terminar limpiamente con exit 0
```

Glob Bomb (Seguridad)

```
python pygrep.py "x" "**/*.log"
# Si hay >50.000 archivos: imprime advertencia y continúa con los primeros 50.000
```

Cobertura de Tests Sugerida

- ☐ Patrones regex básicos (., *, +, ?, ^, \$)
- ☐ Smart-case -S con patrón todo minúsculas (debe actuar como -i)
- ☐ Smart-case -S con patrón con mayúscula (debe ser estricto)

- ☐ Combinación `-i -S` (debe primar `-i`)
- ☐ Patrones con `-F` (fixed strings)
- ☐ Patrones con `-w` (word boundary)
- ☐ Combinación `-F -w`
- ☐ Múltiples patrones (`|` y `-f`)
- ☐ Contexto `-A, -B, -C`
- ☐ Contexto solapado (sin separador `--`)
- ☐ Archivos binarios
- ☐ Encodings no-UTF8
- ☐ Directorios recursivos
- ☐ Comodines `*, ?, []`
- ☐ stdin (pipe)
- ☐ Errores (permisos, archivos inexistentes)
- ☐ Broken pipe (`| head`)
- ☐ ReDoS detection (advertencia y modo strict)
- ☐ Límite de contexto (MAX_CONTEXT_LINES)
- ☐ Glob bomb (MAX_GLOB_EXPANSION)
- ☐ Symlinks ignorados
- ☐ Archivos ZIP, GZ, BZ2, XZ (`-z`)
- ☐ Extract/only-matching (`-o`)
- ☐ Multilinea (`-U`)
- ☐ `--tail` (número positivo, cero, negativo)
- ☐ `--tail` con contexto (`-C`)
- ☐ `--time-range` ISO 8601, Apache, syslog
- ☐ `--time-range` líneas sin timestamp (excluidas)
- ☐ `--time-range` inicio > fin (exit 2)
- ☐ `--time-range` timestamp inválido (exit 2)
- ☐ Combinación `--tail + --time-range`
- ☐ Summary (`--summary`)
- ☐ Stats (`--stats`)
- ☐ Unique (`-u`)
- ☐ Include/exclude (`--include, --exclude`)
- ☐ Ignore dir (`--ignore-dir`)
- ☐ Output fichero (`--output + --format`)
- ☐ Safe print en CP1252

Changelog

v1.5.0

- ☒ `-S / --smart-case`: Búsqueda inteligente de mayúsculas/minúsculas. Si el patrón es todo minúsculas actúa como `-i`; si contiene alguna mayúscula la búsqueda es estricta. La opción `-i` tiene prioridad sobre `-S` cuando se combinan.

v1.4.0 (Modularizada)

-  **Refactorización modular**: División del script original de 1,300 líneas en módulos de bajo acoplamiento bajo la raíz `pygrep_core`.

-  **Corrección de exportador (--output):** El formato de salida de texto ahora respeta la directiva de ocultación de nombres de archivo `-h / --no-filename`.
-  **--tail N:** Muestra solo las últimas N coincidencias en el flujo del patrón.
-  **--time-range:** Permite filtrar registros según intervalos de tiempo (ISO 8601, Apache y syslog).
-  **--follow:** Seguir archivos en tiempo real (similar a `tail -f`) con detección y actualización dinámica del búfer de lectura de Python y soporte de número de línea real.
-  **Ajustes de warnings en Python 3.12:** Captura y silencia avisos en `strptime` sobre formatos syslog.

v1.3.0-hardened (2026-05)

- ☒ **-z / --search-zip:** Búsqueda en archivos comprimidos (`.zip`, `.gz`, `.bz2`, `.xz/.lzma`) con procesamiento streaming.
- ☒ **-o / --only-matching / --extract:** Muestra solo los segmentos de texto que coinciden con el patrón (un match por línea).
- ☒ **--summary:** Tabla de distribución de niveles de log (`ERROR`, `WARN`, `INFO`, `DEBUG`, `FATAL`, `CRITICAL`) con alineación fija y porcentajes.
- ☒ **-U / --multiline:** Búsqueda de patrones que abarcan múltiples líneas mediante lectura completa del stream y mapeo con `bisect`.
- ☒ **OutputEvent.match_span:** Campo para colorear con precisión sub-línea los matches multilinea.
- ☒ **MatchStats.log_level_counts:** Acumulador de niveles de log para `--summary`.
- ☒ **safe_print UTF-8 fallback:** Compatibilidad con terminales Windows CP1252 para caracteres Unicode (box-drawing).
- ☒ Symlinks ignorados en descubrimiento de archivos.
- ☒ Límite de Glob expansion (`MAX_GLOB_EXPANSION = 50 000`).
- ☒ Sanitización de secuencias ANSI en contenido leído.

v1.2.0

- ☒ **--stats:** Estadísticas de búsqueda al finalizar (archivos, líneas, matches, tasa, tiempo).
- ☒ **-u / --unique:** Eliminar líneas duplicadas en los resultados.
- ☒ **--include / --exclude:** Filtrado por patrón de nombre de fichero.
- ☒ **--ignore-dir:** Excluir directorios por nombre.
- ☒ **--output / --format:** Exportación a texto, JSON o CSV.

v1.0.0

- ☒ Implementación inicial con todas las características de grep básico.
- ☒ Soporte completo de contexto `-A/-B/-C` con fusión de grupos.
- ☒ Protección ReDoS con heurística.
- ☒ Corrección TOCTOU en detección de binarios.
- ☒ Tipado estricto mypy-compatible.
- ☒ Cross-platform (Unix + Windows).
- ☒ Expansión nativa de comodines.
- ☒ Búsqueda recursiva con control de profundidad.
- ☒ Soporte de encodings múltiples.
- ☒ Colores ANSI configurables.

Licencia

Este proyecto es de código abierto bajo licencia MIT.

Autores y Contribuyentes

Desarrollado como ejercicio de ingeniería de software con enfoque en:

- Seguridad por diseño (10 vectores de ataque mitigados).
- Tipado estricto (`mypy --strict`).
- Rendimiento y eficiencia de memoria.
- Compatibilidad con estándares POSIX/GNU grep.